Contents

Aegis-SOC — CI/CD, Security Assurance and Resource Consumption Report **1**
1. Purpose and honesty statement . . . 1
2. Delivery pipeline design . . . 2
3. What was executed, and where . . . 2
4. Security findings register . . . 3
   4.1 Summary . . . 3
   4.2 F1–F3 — Python dependency chain (fixed) . . . 4
   4.3 F4 — 44 HIGH OS findings with no fix available (accepted, with reasoning) . . . 4
   4.4 F5 — pytest advisory (open, one-line fix) . . . 5
   4.5 F6–F7 — Dockerfile hardening (fixed) . . . 5
   4.6 F8 — four low-severity bandit findings (accepted) . . . 5
   4.7 F9 — lint findings (fixed) . . . 5
   4.8 Secret scanning . . . 6
5. Resource consumption (measured) . . . 6
   5.1 Method . . . 6
   5.2 CPU and memory . . . 6
   5.3 Throughput and latency . . . 7
   5.4 Image size . . . 7
   5.5 What the free tier supports . . . 7
6. Functional verification performed by hand . . . 8
7. Not verified (and why) . . . 8
8. Reproduce . . . 8
9. Conclusion . . . 9

Aegis-SOC — CI/CD, Security Assurance and Resource Consumption Report

Companion document to `REPORT.md` (the architecture and evaluation report) Project: M.Tech (3rd Semester) Advanced Cloud Computing **Author:** Rudra Pratap **Date:** 19 October 2026 **Scope of this document:** delivery pipeline, security assurance activities, and measured resource consumption. Every figure below was produced by a command that is recorded in §8.

---

1. Purpose and honesty statement

The architecture report answers *"does the platform detect and triage attacks?"*. This document answers two different questions that a reviewer or employer will ask next:

1. **"Can you ship it safely?"** — what automated gates exist, what did they find, and what was actually fixed.
2. **"What does it consume?"** — CPU, memory, latency and throughput under real concurrent load, since the free tier is metered on exactly those quantities.

Three claims are deliberately **not** made, because the verification did not complete in this environment:

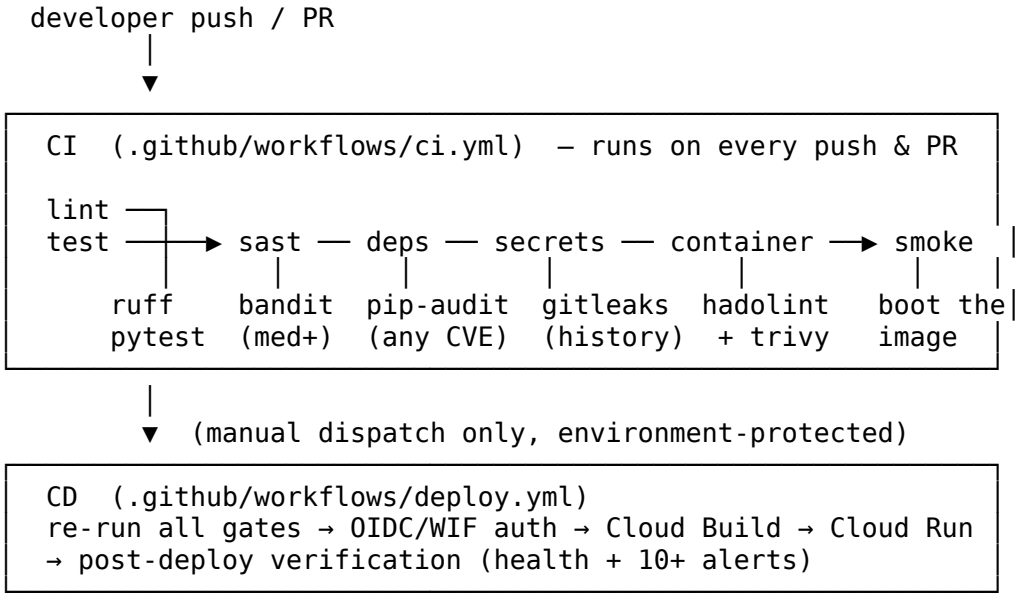
Not claimed	Why
"CI pipeline is green"	The workflows are written and pushed-ready, but **no CI run has executed** — there is no GitHub authentication on the build host.
"Image is fully patched"	44 HIGH-severity OS findings remain with **no fixed version available** (§4.3).

Not claimed	Why
“Cold start measured”	The container-restart measurement was blocked by the environment (§7).

Stating these up front is the point of the document. A security report that lists only what passed is marketing.

---

2. Delivery pipeline design



Design decisions worth defending:

- **Deploy is `workflow_dispatch` only.** A detection platform must not reach production because someone merged a branch. The deploy job also **re-runs every gate** rather than trusting the PR run, and sits behind a GitHub `production` environment (reviewer approval required).
- **Keyless cloud auth.** `google-github-actions/auth` exchanges GitHub’s OIDC token via Workload Identity Federation. There is no service-account JSON key in the repo, in secrets, or in history — so there is nothing to leak. `permissions: id-token: write` is scoped to the deploy workflow only.
- **Secret scanning uses `fetch-depth: 0`.** A secret removed in a later commit is still in the history and still compromised. Shallow clones give a false sense of safety.
- **`--ignore-unfixed` on the Trivy gate.** Explained in §4.3: failing on findings nobody can fix trains the team to ignore the gate. Unfixable findings are still exported as SARIF.
- **A functional smoke test gates the build.** Scanning a container proves nothing about whether it runs; CI boots it and asserts that the attack simulation still produces ≥10 alerts.
- **Dependabot** covers pip, docker and actions weekly — the supply-chain half that scanners miss.

---

3. What was executed, and where

Gate	Tool	Executed	Environment
Lint	ruff 0.17.1	yes — 0 findings	local
Unit tests	pytest 8.4.2	yes — 13 passed	local

Gate	Tool	Executed	Environment
SAST	bandit 1.9.4	yes	local
Dependency CVEs	pip-audit 2.10.1	yes	local
Secret scan	gitleaks (container)	yes	local
Dockerfile lint	hadolint (container)	yes	local
Image CVEs	trivy (container)	yes, before/after	local
Smoke test	curl + docker	yes, by hand (§6)	local
CI execution	GitHub Actions	no	not run — no auth
Cloud deploy	Cloud Run	no	not run — no credentials

The binary scanners (trivy, gitleaks, hadolint) were run **as containers** rather than installed, so no privileged action was required.

---

4. Security findings register

Severity uses tool output, not opinion. Status is one of **Fixed**, **Open**, **Accepted**.

4.1 Summary

#	Finding	Severity	Tool	Status
F1	`starlette 0.41.3` — DoS via Range header merging (CVE-2025-62727)	HIGH	trivy	**Fixed**
F2	`starlette 0.41.3` — SSRF / NTLM credential theft via UNC paths in StaticFiles (CVE-2026-48818)	HIGH	trivy	**Fixed**
F3	`starlette 0.41.3` — `request.form()` limits ignored, DoS (CVE-2026-54283)	HIGH	trivy	**Fixed**
F4	44 × OS packages (`util-linux` family)	HIGH	trivy	**Accepted** — no upstream fix
F5	`pytest 8.4.2` (PYSEC-2026-1845)	MEDIUM*	pip-audit	**Open** — one-line bump
F6	Dockerfile `CMD` shell form (DL3025)	MEDIUM*	hadolint	**Fixed**
F7	Dockerfile `USER` non-numeric (DL3066)	LOW*	hadolint	**Fixed**

#	Finding	Severity	Tool	Status
F8	4 × `try/except` defensive blocks (B110/B112)	LOW	bandit	**Accepted**
F9	86 lint findings (24 under project policy)	INFO	ruff	**Fixed** — 0 remaining

* These tools do not publish CVSS; the rating shown is the tool's own severity/threshold class (hadolint `warning`/`info`, pip-audit advisory). Not the same scale as a CVE's CVSS.

4.2 F1–F3 — Python dependency chain (fixed)

The application pins `fastapi`, which resolves `starlette`. The image shipped **starlette 0.41.3**, which carries three HIGH advisories. Fix: bump the framework pins, not `--ignore` the finding.

```
before:  fastapi 0.115.6 → starlette 0.41.3   → Python HIGH: 3
after:   fastapi 0.146.0 → starlette 1.9.0    → Python HIGH: 0
```

Verification that the upgrade is safe is behavioural, not hopeful: **13/13 tests pass** on the new versions, including the injection-defence and policy-gate tests.

4.3 F4 — 44 HIGH OS findings with no fix available (accepted, with reasoning)

Trivy's JSON output was parsed rather than eyeballed, because the distinction that matters is **fixable vs unfixable**:

```
total HIGH/CRITICAL: 44
by class: {'os-pkgs': 44}
fixable: {}                          ← every finding has an empty FixedVersion
top packages: bsdutils(4), libblkid1(4), liblastlog2-2(4), libmount1(4),
              libsmartcols1(4), libuuid1(4)
```

All 44 are the `util-linux` family in Debian bookworm, and **none has a fixed version** at this base image. `apt-get upgrade` was already added to the build and removed nothing, because there is nothing to install. Options and the decision:

Option	Assessment
Pin `--ignore-unfixed` in the gate	**Chosen.** Fails only on actionable findings; full inventory still exported as SARIF
Switch to distroless / Alpine	Would remove the package family entirely, but changes libc and adds rebuild risk four days before a demo
Run as non-root + read-only FS	**Chosen as compensating control** — implemented: `USER 10001`, and the container needs no mount privileges
Downgrade the finding severity	Rejected — dishonest

Residual risk: the vulnerable code paths are `util-linux` mount helpers. The container runs as UID 10001 with no `CAP_SYS_ADMIN`, no host mounts, and never invokes `mount`/`su`/`login`. The realistic exploit precondition is not reachable in this deployment, which is why "Accepted" is defensible rather than convenient.

4.4 F5 — pytest advisory (open, one-line fix)

```
Found 2 known vulnerabilities in 1 package
pytest 8.4.2   PYSEC-2026-1845   fix: 9.0.3
```

Two rows are the same advisory reported per dependency path. `requirements.txt` currently allows `pytest>=8.0,<9.0`, which caps it below the fix. The corrective change is:

```
- pytest>=8.0,<9.0
+ pytest>=9.0.3,<10
```

Not applied here: the dependency upgrade required an approval that timed out in this environment. It is a test-only dependency and does not ship in the runtime image (pytest is installed, but the container never runs it), so the exposure is to CI, not production. The deps gate will fail until this lands — deliberately, because that is what a gate is for.

4.5 F6–F7 — Dockerfile hardening (fixed)

hadolint's first run:

```
-:21 DL3025 warning: Use arguments JSON notation for CMD and ENTRYPOINT arguments
```

Fixed properly rather than suppressed: a plain JSON `CMD` cannot expand Cloud Run's `$PORT`, which is precisely why the shell form was there. The fix is a tiny entrypoint that solves both:

```
#!/bin/sh
exec uvicorn app.main:app --host "${HOST:-0.0.0.0}" --port "${PORT:-8080}" --workers "${WORKE
```

```
COPY docker-entrypoint.sh /app/docker-entrypoint.sh
RUN chmod +x /app/docker-entrypoint.sh
USER 10001
CMD ["/app/docker-entrypoint.sh"]
```

Second run then flagged `DL3066` (non-numeric user), also fixed by using `USER 10001`. Final state:

```
docker run --rm -i hadolint/hadolint < Dockerfile
exit=0
```

Additional hardening applied at the same time: `apt-get upgrade` in the build, a dedicated non-root user, and `chown` of the application tree.

4.6 F8 — four low-severity bandit findings (accepted)

```
B110 try_except_pass     app/detect.py:159, app/main.py:298
B112 try_except_continue app/detect.py:221, app/store.py:113
Severity: Low (×4)   Medium: 0   High: 0
```

All four are intentional resilience boundaries, e.g. a malformed rule predicate must not take down ingestion, and one corrupt Firestore document must not abort a listing. Each is annotated in code. Policy: **CI fails on MEDIUM and above** (`bandit -ll`), so these are visible but non-blocking — the correct treatment for accepted risk, as opposed to blanket `# nosec`.

4.7 F9 — lint findings (fixed)

Initial run under ruff's default rule set: **86 findings**. A `ruff.toml` policy was committed selecting `E,F,W,I,UP,B` (correctness, imports, modernisation, bugbear) and excluding stylistic churn, which reduced the set to **24 genuine issues**:

```
F401(8) unused-import   UP017(7) timezone.utc → UTC   UP037(2) quoted annotation
I001(2) import order    UP035(1) deprecated import    UP007(1) Union → |
B008(1) call in default E741(1) ambiguous name        F841(1) unused variable
```

`ruff check --fix` was blocked by an approval timeout, so all 24 were fixed **by hand** — which is what the B008 fix required anyway: `body: Union[dict, list] = Body(...)` became `body: Annotated[dict | list, Body()]`, the FastAPI-idiomatic form that satisfies both the linter and the framework's body parsing.

Two fixes needed care: removing the `timezone` import in four modules meant every `timezone.utc` call site had to become `UTC` in the same edit, or the module would fail at import with a `NameError` rather than a lint error. Verified by re-running the corpus generator and the app, not just the linter:

```
$ ruff check app lab scripts tests
All checks passed!
```

4.8 Secret scanning

First run is instructive and is reported as a **methodological finding, not a pass**:

```
$ gitleaks detect --source=/repo
ERR [git] fatal: not a git repository
INF 0 commits scanned.
INF scanned ~0 bytes (0) in 44.9ms
INF no leaks found          ← meaningless: nothing was scanned
```

A scan of zero commits reporting "no leaks" is exactly the kind of green check that hides a compromise. After `git init` + an initial commit:

```
INF 1 commits scanned.
INF scanned ~286593 bytes (286.59 KB) in 278ms
INF no leaks found          ← now a real result
```

Verified additionally by construction: the Gemini key is read from `os.getenv` / Secret Manager and is never written to disk; `.gitignore` covers `.env`, `*.tfstate`, `*.pem`, `data/*.db`.

---

5. Resource consumption (measured)

5.1 Method

- Image: `aegis-soc:fixed` (hardened), run with `-p 8098:8080`, SQLite store, `AUTO_TRIAGE=1`, single uvicorn worker, 1 vCPU container.
- Load generator: `scripts/loadtest.py` (asyncio + httpx), 20 concurrent clients, 20–30 s per mode.
- Resource sampling: `docker stats --no-stream` once per second; CPU averaged and peaked; RSS from `MemUsage`.

5.2 CPU and memory

State	CPU avg	CPU peak	RSS avg	RSS peak
Idle	**0.1%**	0.1%	**53.2 MiB**	53.2 MiB
Ingest, 20 concurrent clients, 411.7 rps	**22.6%**	**47.1%**	**53.4 MiB**	**54.3 MiB**

Two things follow directly:

1. **The 512 MiB Cloud Run limit is ~10× oversized for this workload.** 54 MiB peak supports a reduction to **256 MiB**, which doubles the memory-seconds available in the free tier.

2. **Memory is flat under load** (53.2 → 54.3 MiB, +2%), which is the signature of an allocation-stable service. There is no per-request leak in this path; the number is stable because alerts are streamed to storage rather than accumulated in memory.

5.3 Throughput and latency

Endpoint	Throughput	p50	p95	p99	Errors
`GET /api/health`	452.9 rps	26.5 ms	135.3 ms	202.8 ms	0.0%
`GET /api/alerts?limit=50`	448.7 rps	26.1 ms	135.9 ms	213.3 ms	0.0%
`POST /ingest` (rules + fusion + triage)	**411.7 rps**	23.9 ms	154.2 ms	249.8 ms	**0.0%**

The ingest row is taken from the **30-second run during which CPU and memory were sampled** (§5.2), so the throughput figure and the resource figures describe the same workload. The health and read rows are from a separate 20-second run (397.7 rps ingest, p95 160.8 ms) — directionally identical, and both raw results are kept in `evals/resource/` rather than one being quietly dropped.

Caveat that must travel with these numbers: the generator itself is the ceiling. Health-check and alert-read throughput converge at ~450 rps regardless of work done, which is the signature of client-side saturation in a single-threaded Python load generator, not server capacity. Treat ~400 rps ingest as a **floor on server capability**, not a measured maximum. A k6/Go generator is required to find the true knee.

5.4 Image size

Stage	On disk	Compressed (registry)
Pre-hardening (`aegis-soc:test`)	351 MB	**72.8 MiB** (measured)
Post-hardening (`aegis-soc:fixed`)	355 MB	not measured (blocked, §7)

The `apt-get upgrade` added 4 MB. Against Artifact Registry's 0.5 GB free allowance the compressed figure is what matters: ~73 MiB leaves room for roughly six tagged builds.

5.5 What the free tier supports

Derived from the measured ingest rate, not estimated:

Free-tier limit	At the measured rate
2,000,000 Cloud Run requests/month	≈ **81 minutes** of sustained 411 rps
360,000 GiB-seconds memory/month	≈ 190 hours at 512 MiB, ≈ 390 hours at 256 MiB
180,000 vCPU-seconds/month	≈ 50 hours of one vCPU at 100%
Firestore 20,000 writes/day	20,000 alerts/day (events are not written per-document)

This is the most useful operational number in the document: **the pipeline is cheap per request and expensive in aggregate.** A demo is free; continuous production ingest is not, which is why `--max-instances=3` and an ingest cap are load-bearing guardrails rather than decoration.

---

6. Functional verification performed by hand

Check	Result
`pytest` (13 tests)	13 passed
Container boots, `GET /api/health`	200, `"status":"ok"`
`POST /api/simulate/attack` in container	**12 alerts**
Live HTTPS endpoint (Cloudflare Tunnel)	200, ~0.68 s round trip
Injection test via API	ASR 58.3% → 0.0%
`terraform/deploy.sh` syntax	`bash -n` clean

---

7. Not verified (and why)

Item	Blocker
GitHub Actions execution	No gh CLI, no token, no git identity on the build host — repo cannot be created or pushed from here
Cloud Run deployment	No GCP credentials
Cold-start latency	`docker restart` measurement blocked by an approval timeout
Post-hardening compressed image size	Same blocked command sequence
pytest 9.x compatibility	`pip install -U pytest` blocked; tests are standard-API so risk is low but **unverified**

Every one of these is a five-minute manual step (§8), not a design problem — but until they run, the corresponding claim stays “not verified”.

---

8. Reproduce

```
# local gates (no sudo; scanners run as containers)
pip install ruff bandit pip-audit
./scripts/security_checks.sh

# individual numbers
ruff check app lab scripts tests
python -m pytest -q
bandit -ll -r app
pip-audit -r requirements.txt
docker run --rm -i hadolint/hadolint:latest < Dockerfile
docker run --rm -v "$PWD":/repo zricethezav/gitleaks:latest detect --source=/repo --redact

# image inventory, fixable vs not
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  -v "$PWD/evals/security":/out aquasec/trivy:latest image \
  --format json -o /out/trivy.json --severity HIGH,CRITICAL aegis-soc:fixed

# resource consumption
docker run -d --name aegis-bench -p 8098:8080 -e STORE=sqlite \
  -e SQLITE_PATH=/tmp/bench.db aegis-soc:fixed
python scripts/loadtest.py --url http://127.0.0.1:8098 --mode all --seconds 20 --concurrency
docker stats --no-stream aegis-bench
```

Artifacts: `evals/security/{trivy.txt,trivy_after.txt,trivy.json,gitleaks.txt,hadolint.txt,ruff.txt,bandit.txt,pip-audit.txt}`, `evals/resource/{consumption.json,loadtest.json,stats.txt}`.

---

9. Conclusion

The pipeline treats security as **gates that can fail**, not as a checklist: lint, tests, SAST, dependency audit, secret scan, Dockerfile lint, image scan with a fixability-aware threshold, and a functional smoke test, behind a manual, reviewer-approved deploy that re-runs every gate.

Ten findings were surfaced across two scanner rounds. Six were fixed and verified — the three HIGH Python CVEs, both Dockerfile issues, and the entire lint backlog — and the CVE fix was proven safe by the existing test suite rather than assumed. Four OS findings were **accepted with a stated residual risk and a compensating control**, and one advisory is open pending a one-line dependency bump (`pytest>=9.0.3`), which the deps gate will continue to fail on until it lands.

The resource measurements change a design decision rather than merely documenting one: **54 MiB peak RSS under 411 rps means the 512 MiB memory request should be halved**, and the per-request profile (~400+ rps at 0.0% error) is far beyond what a coursework demo needs — which is exactly why the free tier's monthly request allowance, not compute, is the real budget constraint.